

CS744 Course Project: Multithreaded Server Design

A Concurrent, Sandboxed C Auto-Grading Server

Vansil Chauhan
24M0847

August 15, 2026

Overview

This project is a client/server auto-grader for C programs. A TCP server compiles a reference solution once at startup, precomputes its expected output on a fixed set of test inputs, and then accepts student submissions over the network: for each submission it compiles the student's file, executes the resulting binary against every test case, compares the output against the precomputed answer key, and reports back a pass percentage. A companion TCP client streams a student's `.c` file to the server and prints whatever result comes back.

The interesting engineering problem the project poses is not the grading logic itself — comparing strings is trivial — but everything around it: how to grade many submissions concurrently without one slow submission blocking every other student, and how to safely compile and run *arbitrary, untrusted* code on a shared server without letting one malicious or buggy submission take the whole grading service down. Those two concerns, concurrency and isolation, are what most of the code and most of this report are about.

The project was originally built for the course in 2024 and revisited in August 2026 to fix a build-breaking bug, add a real sandbox around student code execution, and run a systematic performance evaluation of the concurrent design. All three pieces — the original server, the sandbox, and the performance study — are treated here as one project, since the sandbox and measurements are a direct extension of the same codebase and the same design questions.

What was built

The repository contains:

- `server.c` — the grading server: a thread pool over a hand-rolled producer/consumer task queue, a compile-and-run pipeline built on `fork/exec`, and a sandboxing layer (resource limits, execution timeouts, upload caps) around every piece of untrusted code it runs.
- `client.c` — a minimal TCP client that streams a file to the server and prints the result string it gets back.
- `solution.c`, `program_1.c` through `program_4.c` — a reference solution and four sample "student" submissions of varying correctness, used both for manual testing and as the load-generation workload.
- `test_cases.txt` — one input per line, fed as `argv[1]` to both the reference solution and every student submission.
- `runserver.sh` / `runclient.sh` — convenience scripts to start the server and to fire `N` concurrent client submissions via `xargs -P 10`.
- `bench/` — a purpose-built Python benchmarking harness (`bench_client.py`, `harness.py`, `run_experiments.py`, `make_plots.py`) used to run and plot the performance experiments in this report.
- `results/` — raw CSVs, generated plots, and a standalone performance write-up (`results/report.md`) that this report's evaluation section draws on directly.

Functionally the system does exactly one thing: grade a C submission against a fixed set of test cases and return a percentage. All of the engineering complexity is in doing that concurrently, safely, and with measured (not assumed) performance characteristics.

How it works

Architecture

The server is a single process. One main thread accepts TCP connections; a fixed-size pool of worker threads, created once at startup, does all of the actual grading work. The two are connected by a shared, mutex- and condition-variable-protected task queue: the accept loop is the *producer*, the worker threads are *consumers*.

Server startup

`main()` (`server.c`, lines 594–680) does five things in order: parse `<port>` `<thread_count>` `<solution.c>` `<test_cases.txt>` from `argv`; compile `solution.c` into `solution.o`; read `test_cases.txt` line by line and, for each line, run the compiled reference solution with that line as `argv[1]`, storing its stdout as the expected answer for that test case; initialize the queue's mutex and condition variable and spawn `thread_count` worker threads (all running `thread_work`, all immediately blocked waiting for work); and finally enter an infinite `accept()` loop. The answer key is therefore computed exactly once, up front, rather than being recomputed per submission — every subsequent grading pass is just a string comparison against those precomputed outputs.

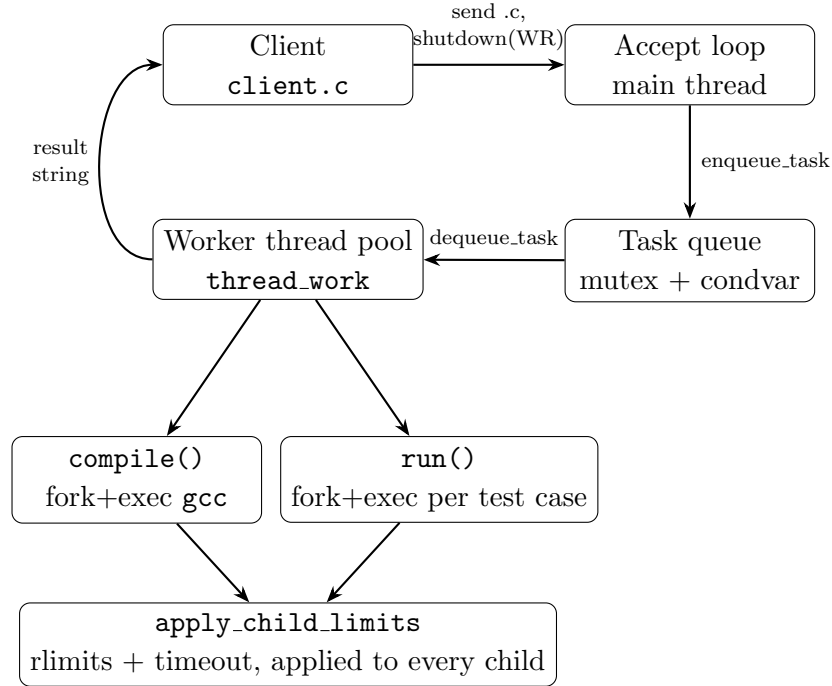


Figure 1: One accept-loop thread hands off every connection to a shared task queue; a fixed pool of worker threads pulls sockets off that queue and does the compile/run/compare work, with every child process constrained by `apply_child_limits` before exec.

The task queue

The queue (`server.c`, lines 312–371) is a hand-rolled singly linked list guarded by `queue_lock` and signaled with the `task_available` condition variable:

Listing 1: Producer and consumer sides of the task queue (`server.c`)

```

1 void enqueue_task(int *arg) {
2     TaskNode *new_node = create_task_node(arg);
3     pthread_mutex_lock(&queue_lock);
4     if (queue.rear == NULL) queue.front = queue.rear =
5         new_node;
6     else { queue.rear->next = new_node; queue.rear = new_node
7         ; }
8     queue.size++;
9     pthread_cond_signal(&task_available);
10    pthread_mutex_unlock(&queue_lock);
11 }
12
13 int *dequeue_task() {
14     pthread_mutex_lock(&queue_lock);
15     while (queue.size == 0)
16         pthread_cond_wait(&task_available, &queue_lock);
17 }

```

```

15     TaskNode *temp = queue.front;
16     int *socket = temp->socket;
17     queue.front = queue.front->next;
18     if (queue.front == NULL) queue.rear = NULL;
19     free(temp);
20     queue.size--;
21     pthread_mutex_unlock(&queue_lock);
22     return socket;
23 }

```

Two details matter here beyond the basic locking: the wait is a `while`, not an `if`, which is what makes it safe against spurious wakeups (a thread that wakes up must re-check the condition before trusting it); and the signal happens while still holding the lock, so there is no window where a signaled thread wakes up to find the queue already empty again. `thread_count` is, quite literally, how many submissions can be graded in parallel — `runserver.sh` sets it to 1000, a value the performance evaluation in this report shows to be badly oversized (Section).

Grading one submission

Once a worker pulls a socket off the queue, `thread_work` (`server.c`, lines 472–571) does the following per connection:

1. Picks a unique filename of the form `student_program_<timestamp>_<client_number>.c`, where `client_number` comes from a mutex-protected global counter (`get_client_count`) — this is what stops two concurrent submissions from colliding on the same temp filename.
2. Reads the socket in a loop and writes every received chunk straight into that file, using the exact byte count `recv` returns (not a NUL-terminated `%s` write, since the buffer can legitimately contain no terminator) until the client closes its write side. This is why `client.c` calls `shutdown(sock, SHUT_WR)` after sending: that is what makes `recv` return 0 on the server side, ending the loop. The loop also enforces a 1 MB cap on total upload size (Section).
3. Compiles the file via `compile()`, the same fork-and-run-gcc pattern used for the reference solution.
4. For each precomputed test case, calls `run(executable, input)`: forks, redirects the child’s stdout into a pipe, `execvs` the compiled student binary with the test input as `argv[1]`, and captures whatever it prints.
5. Compares each captured result against the matching precomputed solution output and tallies a `success_count`.
6. Formats a result string such as `12.34% (1/8) test cases passed!` and writes it back on the socket.
7. Deletes the temporary `.c` and `.o` files and closes the socket.

`run()` does double duty: it is used once per line of `test_cases.txt` during startup

to build the answer key, and once per test case per submission during grading.

The client

`client.c` is deliberately simple: connect, stream the given file into the socket via `send_file` (line by line, using `fgets/send`), call `shutdown(sock, SHUT_WR)` to signal "done sending," block on a single `recv()` for the result string, and print it. `runclient.sh <N>` fires `N` such submissions, cycling through `program_1.c..program_4.c` as "students," 10 at a time in parallel via `xargs -P 10`.

Sandboxing untrusted code execution

The server's core function — compiling and running whatever C a client sends it — is also its biggest liability. The original 2024 version ran that code with no resource limits, no execution timeout, and no upload cap: an infinite loop in a submission would hang a worker thread forever, a fork bomb would run unchecked, and there was nothing stopping a malicious client from writing gigabytes to disk. The August 2026 revisit closed these gaps directly in `server.c`, and — because the learning objective was understanding the mechanism, not just citing that limits exist — verified the result by attacking the server with five hostile submissions rather than by code inspection alone.

What changed

- `apply_child_limits()`, called in every child between `fork()` and `exec()`. It calls `setsid()` first, so the child becomes the leader of its own process group; that is what later allows `kill(-pid, SIGKILL)` to take down the entire descendant tree of a submission that forks before hanging, rather than leaving orphans behind. It then applies `RLIMIT_CPU`, `RLIMIT_AS`, `RLIMIT_FSIZE`, `RLIMIT_CORE=0`, `RLIMIT_NOFILE`, and `RLIMIT_NPROC`. Limits differ for the two kinds of child the server ever runs, shown in Table 1: a student binary is capped tightly, while `gcc` is given a looser budget and its process count is deliberately left uncapped, since `gcc` legitimately forks `cc1`, `as`, and `ld` as part of a normal compile — capping `NPROC` there would break the compiler, not stop an attacker.
- `wait_timeout()` enforces the wall-clock deadline: a `waitpid(WNOHANG)` poll loop watches for the child to exit, and on expiry sends `SIGKILL` to the child's process group, then to the child itself, then does a final blocking reap.
- `capture_with_timeout()` drains the child's stdout pipe with `poll()` while simultaneously watching the deadline, rather than waiting for the child to exit first and reading the pipe afterward. This also fixes a latent deadlock in the original code: if a program printed more than one pipe buffer's worth of output, the child would block in `write()` on a full pipe while the server blocked in `waitpid()`, permanently losing that worker thread. Output beyond the capture

cap is now read and discarded rather than left to fill the pipe.

- A **1 MB cap on uploaded source**, enforced in the `recv` loop in `thread_work`, rejected with an explicit message rather than silently truncated.
- A **killed-test-case marker** (<<killed>>) written into a result slot instead of partial output whenever a test case is killed, chosen so it can never coincide with a legitimate program's output and be graded as a pass.

Limit	Student binary	gcc
RLIMIT_CPU	2 s	10 s
RLIMIT_AS (address space)	256 MB	1 GB
RLIMIT_FSIZE (file writes)	8 MB	64 MB
RLIMIT_NOFILE (open fds)	64	unlimited
RLIMIT_NPROC (processes)	64	<i>uncapped, deliberately</i>
Wall-clock timeout	3 s	15 s

Table 1: Resource limits applied by `apply_child_limits()`, defined as `RUN_* / COMPILE_*` constants in `server.c`. `gcc`'s process count is left uncapped because it legitimately forks `cc1`, `as`, and `ld`; capping it would break compilation itself.

The two functions below show how these limits are actually wired in. `apply_child_limits` runs in the child, strictly between `fork()` and `exec()` — limits set any later would be too late, since the untrusted program would already be running:

Listing 2: Applying resource limits to a child before `exec` (`server.c`)

```

1 static void apply_child_limits(int cpu_sec, rlim_t as_bytes,
2                               rlim_t fsize,
3                               int nproc, int nofile) {
4
5     if (!sandbox_enabled) return;
6
7     /* New process group => kill(-pid, SIGKILL) can take down
8        the
9        * whole descendant tree, not just this one process. */
10    setsid();
11
12    set_limit(RLIMIT_CPU, cpu_sec);          /* SIGKILL on CPU
13                                             exhaustion */
14    set_limit(RLIMIT_AS, as_bytes);          /* malloc fails
15                                             past this */
16    set_limit(RLIMIT_FSIZE, fsize);          /* SIGXFSZ past
17                                             this */
18    set_limit(RLIMIT_CORE, 0);               /* no core dumps on
19                                             the server */
20
21    if (nofile > 0) set_limit(RLIMIT_NOFILE, nofile);
22    if (nproc > 0)  set_limit(RLIMIT_NPROC, nproc); /* fork
23                                             () fails past this */

```

```
15 }
```

The second function is the deadlock fix described above: draining the pipe and waiting for the deadline happen in the *same* poll loop, so neither can block on the other:

Listing 3: Draining output while racing a deadline, instead of waiting then reading (server.c, abridged)

```
1  for (;;) {
2      int remaining = timeout_ms - (int)(now_ms() - start);
3      if (timeout_ms >= 0 && remaining <= 0) { timed_out = 1;
4          break; }
5
6      struct pollfd pfd = { .fd = readfd, .events = POLLIN };
7      int pr = poll(&pfd, 1, remaining);
8      if (pr == 0) { timed_out = 1; break; }           /* deadline
9                                                         hit */
10
11     ssize_t n = read(readfd, chunk, sizeof(chunk));
12     if (n == 0) break;                               /* child +
13                                                         all fds closed: real EOF */
14     /* ...append to out[], capped at outcap, discard the rest
15        ... */
16 }
17 if (timed_out) { kill(-pid, SIGKILL); kill(pid, SIGKILL); }
18 waitpid(pid, status, 0);
```

Verified by attack

Rather than trust the limits by inspection, five hostile submissions were run against the sandboxed server, each checked for containment and for whether the server kept grading correctly afterward:

Hostile submission	Observed outcome
Infinite loop	SIGKILL on RLIMIT_CPU at ~2s per case
64MB malloc loop	Allocation fails; exits on its own after 2.1s total
Unbounded stdout	Drained without deadlock; SIGKILL on CPU limit
Fork 5 children + sleep(60)	3s wall timeout; zero orphans left alive
100 GB file write	Capped at exactly 8,388,608 bytes (RLIMIT_FSIZE)

Table 2: Five hostile submissions run against the sandboxed server. A baseline honest submission scored 36.36% (4/11) both before and after the whole sequence — identical, confirming no worker threads were lost and grading semantics were unaffected.

Measuring the cost of the sandbox

A natural question is whether the sandbox slows grading down. A `SANDBOX=0` environment toggle was added so the exact same server binary could be run with limits and timeouts disabled, letting the cost be measured against itself on the same machine rather than inferred from an older run. The first attempt at this comparison — lining the newly measured sandboxed numbers up against the *original* 2024 Experiment A data — showed an apparent 15–25% throughput penalty. That result did not survive scrutiny: the two runs were separated by roughly two years and by whatever else was running on the machine at the time, so the “penalty” was measuring drift in the machine as much as the change under test. Re-running both arms back to back (`SANDBOX=0` then `SANDBOX=1`, same session, same load) gave a mean difference of **+3.9%** — inside normal run-to-run variance. This makes sense mechanically: the added cost per submission is a handful of `setrlimit` calls and one `setsid` against 130–190ms of compile time, and a conforming program never approaches any of the ceilings. Both arms independently reproduced the throughput peak at 8 threads from the original experiment, and the sandboxed run additionally eliminated the 512-thread failures the unsandboxed run had shown (1 failure → 0).

The methodological point is the one worth keeping: **a before/after comparison that spans a gap in time is not an A/B test**. Only a same-session, back-to-back comparison isolates the variable actually being measured.

Key decisions and tradeoffs

Process-based isolation via `fork+exec`, not in-process execution. Every piece of untrusted code — the reference solution, every student submission, `gcc` itself — runs in its own child process, never in-process. This is what makes resource limits, timeouts, and process-group kills possible at all; it is also the textbook pattern for “run code I don’t trust” that generalizes directly to CI runners and other grading systems.

Timeout via `pidfd_open`, not a sleep-poll loop. There is no timed `waitpid()` in POSIX. The naive fix — poll `waitpid(WNOHANG)` on a short sleep — was tried first and measured to wake every worker thread roughly 75 times per compile on a 2ms loop, which is exactly the kind of overhead that shows up as reduced throughput under load. `pidfd_open()` makes a child waitable through `poll()`, so a worker blocks until the child exits or the deadline passes, with zero intermediate wakeups; the sleep-loop version is kept only as a fallback for kernels without `pidfd_open`.

A dedicated Python benchmark harness, not `client.o` shelled out repeatedly. The performance experiments needed a load generator whose own overhead didn’t contaminate the numbers being measured. Reusing `client.c` via `xargs -P 10` (as `runclient.sh` does for informal testing) would mix the per-request `fork/exec` cost of spawning `client.o` itself into what was supposed to be a measurement of

server-side latency. `bench/bench_client.py` instead reimplements the client’s wire protocol directly over raw sockets, driven by a Python thread pool, so only the server’s behavior is under test.

Bolted-on timing instrumentation, not a redesign. Attributing time to ”compile” versus ”execute” (Experiment D) needed per-phase timestamps on the server side. Rather than restructure anything, `clock_gettime(CLOCK_MONOTONIC)` calls were added around `compile()` and the test-case loop in `thread_work`, logged to stderr as a `TIMING` line. This touches nothing on the wire protocol or in the grading logic — it is purely an observation point, which made it safe to add without risking a behavior change in the system being measured.

Fix the sandboxing gap before optimizing performance. The codebase supported two directions of follow-up work: making the server faster, or making it safe to run untrusted code at all. The security gap was treated as higher priority, on the reasoning that an unbounded worst-case (an unkillable hung thread, an unbounded upload) is worse than a suboptimal average case (an oversized thread pool). This is reflected in the order of the work log: sandboxing landed before the thread-pool right-sizing recommendation from Experiment A was acted on.

Performance evaluation

Setup

All experiments ran on a single machine (Intel i7-1185G7, 8 logical cores, 16 GB RAM) with client and server co-located over loopback, so there is no network-latency component — all measured time is connection setup plus compile plus execution plus scheduling on one box. Getting the harness running at all required first fixing a pre-existing build break: `thread_work()` was declared as `void *thread_work()` instead of `void *thread_work(void *arg)`, which does not match the signature `pthread_create` expects; a newer `gcc` treats this as a hard error (`-Wincompatible-pointer-types`) rather than a warning, so the project did not compile on this machine until the mismatch was fixed.

Workloads reused the repository’s four sample submissions (`program_1.c`–`program_4.c`), cycled round-robin, held constant across all four experiments so only the variable actually under test changes. `test_cases.txt` (11 inputs) was used for Experiments A and B; synthetically generated files of 5/10/20/50/100 random integer inputs were used for Experiments C and D, since test-case count is itself the independent variable there.

Two dependent variables were measured throughout: **throughput** (submissions graded per second, over a batch’s wall-clock time) and **latency** (end-to-end, client-send-start to result-received, reported as mean/p50/p95/p99/max). Experiment D additionally splits server-side time into compile time versus execution time using the

Exp.	Independent variable	Held constant
A	Thread pool size: 1–512	50 concurrent clients, 11 test cases, 200 subs./config
B	Offered load: 1–400 concurrent clients	256 threads, 11 test cases, 200 subs./config
C	Test cases/submission: 5–100	64 threads, 20 concurrent clients, 100 subs./config
D	Test cases/submission: 5–100 (same runs as C)	64 threads, 20 concurrent clients

Table 3: Independent variables and fixed configuration per experiment.

TIMING stderr log described above.

Experiment A — thread pool size vs. throughput/latency

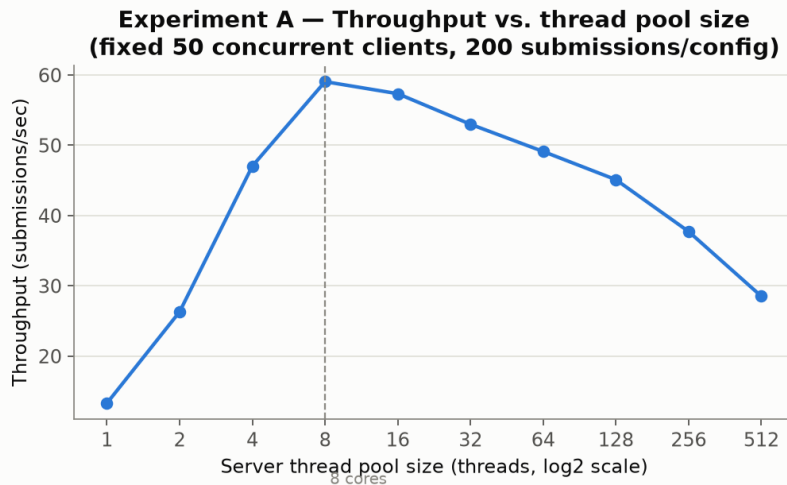


Figure 2: Throughput vs. server thread pool size (1–512 threads).

Throughput rises steeply from 1 to 8 threads (13.3 to 59.0 submissions/sec, $\sim 4.4\times$) and **peaks exactly at 8 threads — matching the machine’s 8 logical cores**. Past 8 threads it declines monotonically, down to 28.5 rps at 512 threads (a $\sim 52\%$ drop from peak). p50 latency bottoms out at 8 threads (810.6 ms) and roughly doubles by 512 threads (1720.3 ms); p95 nearly quadruples (909.8 ms $\rightarrow$ 2932.3 ms). Server CPU utilization climbs monotonically with thread count (2.7% $\rightarrow$ 90.6%) even as throughput falls, meaning the extra threads are spending time context-switching and contending, not doing useful work. At 512 threads, 3 of 200 submissions failed outright.

Each submission is CPU-bound (`fork+exec` of `gcc`, then `fork+exec` per test case) with negligible I/O wait, so throughput is capped by physical core count, not by how

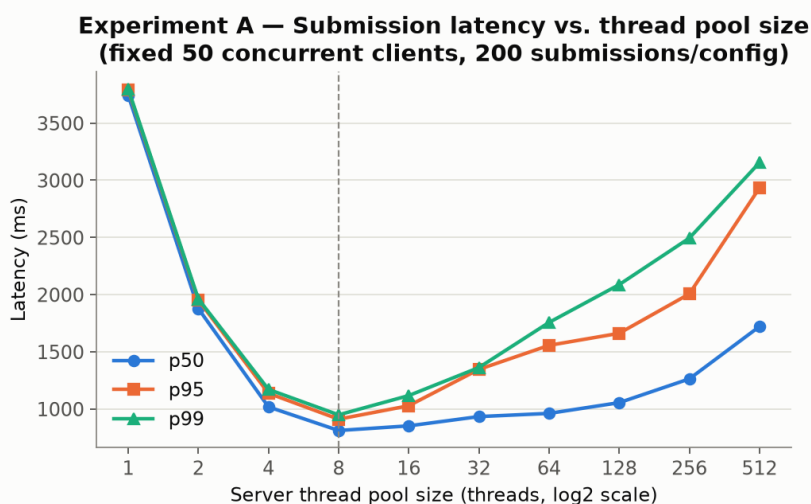


Figure 3: Latency (p50/p95) vs. server thread pool size (1–512 threads).

many threads are available to be scheduled. `runserver.sh`'s default of **1000 threads** is **badly oversized** for this hardware — it lands on the degrading tail of this curve for no benefit; a pool size in the 8–16 range performs at least as well while using far fewer resources. The failures at 512 threads point at resource exhaustion, consistent with many threads each concurrently forking and execing `gcc` and a student binary at once (likely FD or process-count pressure).

Experiment B — offered load vs. throughput/latency

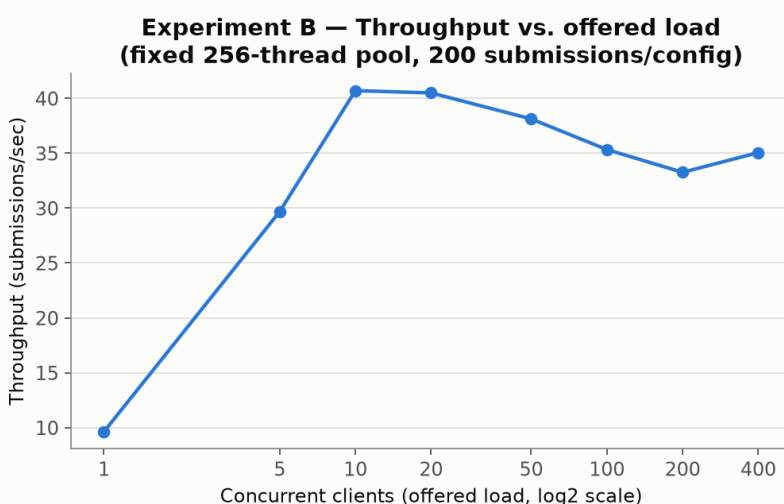


Figure 4: Throughput vs. offered load (1–400 concurrent clients), 256-thread pool.

Throughput rises from 9.6 rps at concurrency 1 to a plateau of roughly 35–41 rps from concurrency 10 onward; it does *not* keep climbing with offered load past that point. p50 latency grows almost linearly with concurrency once past the plateau:

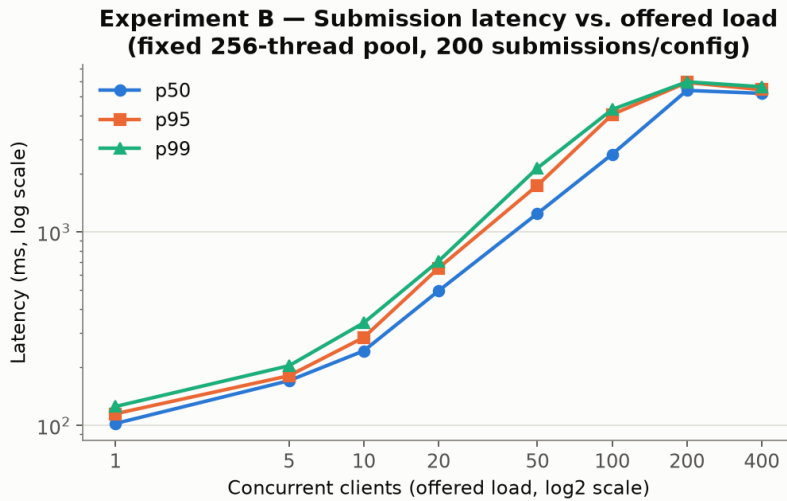


Figure 5: Latency (p50/p95) vs. offered load (1–400 concurrent clients), 256-thread pool.

102 ms (concurrency 1) → 495 ms (20) → 2518 ms (100) → 5384 ms (200) — the classic queueing signature of throughput saturating while extra load simply waits longer. At concurrency 400, throughput does not improve over concurrency 200 (35.1 vs. 33.3 rps) and 9 of 200 requests failed: the system is fully saturated and starting to shed load.

With a 256-thread pool on an 8-core box, the effective concurrency ceiling is set by cores, corroborating Experiment A. More importantly, latency growth here is **unbounded because there is no admission control or backpressure** — `enqueue_task` accepts work indefinitely (bounded only by `malloc` and `listen()`’s backlog of 10), so clients wait rather than getting a fast ”server busy” response. This is exactly the failure mode a request timeout or bounded queue would guard against in a real deployment.

Experiment C — test cases per submission vs. latency

Average latency grows from 320 ms (5 test cases) to 1509 ms (100 test cases) — roughly **4.7× latency for a 20× increase in test-case count**, i.e. sub-linear growth. Throughput at fixed load drops correspondingly, from 61.3 to 12.7 rps across the same range. This is explained by Experiment D below: a large, roughly fixed per-submission cost (compilation) is amortized over more test cases as the count grows. Practically, this is a direct capacity-planning input: courses with larger test suites per assignment should expect proportionally lower grading throughput on fixed hardware.

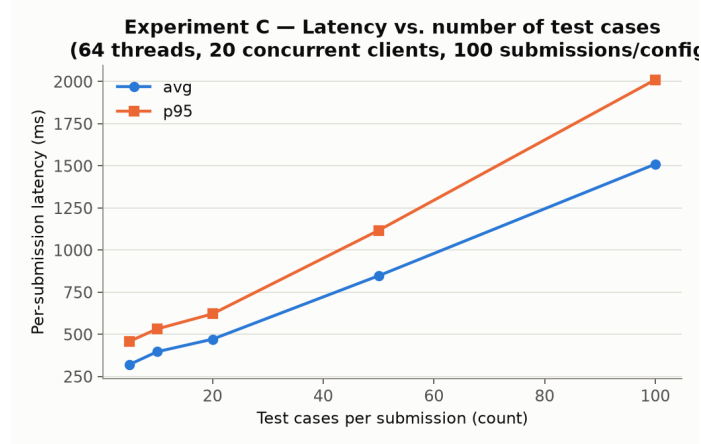


Figure 6: Average submission latency vs. number of test cases (5–100), 64-thread pool, 20 concurrent clients.

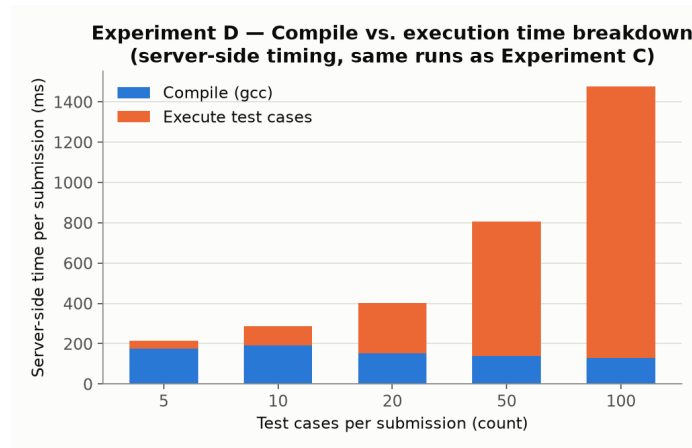


Figure 7: Compile time vs. execution time as a function of test-case count (same runs as Experiment C).

Experiment D — compile vs. execution time breakdown

Compile time (gcc) is roughly constant regardless of test-case count, averaging 130–191 ms across all five configurations, since it depends on source size/complexity rather than how many times the resulting binary is later run. Execution time grows essentially linearly with test-case count: 37.9 ms (5 cases) → 97.3 ms (10) → 250.2 ms (20) → 666.8 ms (50) → 1347.1 ms (100), roughly 13.5 ms per test case on average. At 5 test cases, compile time *dominates* (82% of total time); by 100 test cases, execution dominates instead (91% of total time).

There is a clear crossover: for small test suites, the compile step is the bottleneck, so the highest-leverage optimization there would be caching or speeding up compilation, not parallelizing test execution. For large suites, the current design’s one `fork+exec` per test case (no batching, no persistent process reuse) is the dominant cost, and would be the first thing to optimize if faster grading of large suites were a goal — for

example, a single run of the student binary handling all test inputs, if the assignment contract allowed it.

Summary

Exp.	Key result
A	Throughput peaks at 8 threads (= core count); grows $4.4\times$ from 1→8 threads, then degrades 52% by 512 threads. <code>runserver.sh</code> 's 1000-thread default is oversized.
B	Throughput plateaus at $\sim 35\text{--}41$ rps past ~ 10 concurrent clients; latency grows roughly linearly with offered load beyond that (no backpressure).
C	Latency grows sub-linearly with test-case count ($4.7\times$ latency for $20\times$ test cases) because a large fixed compile cost is amortized over more cases.
D	Compile time is roughly constant ($\sim 130\text{--}190$ ms); execution time grows $\sim$ linearly (~ 13.5 ms/test case). Small suites are compile-bound; large suites (20+ cases) are execution-bound.

Table 4: Overall takeaway: a CPU-bound workload with no I/O-wait to hide behind extra threads, so pool size should track core count rather than be set arbitrarily high; and with no admission control, latency (not failure) is what degrades first as offered load exceeds capacity.

Results and current state

The server, as it stands, compiles and grades cleanly, survives the five hostile submissions in Section without losing a worker thread or corrupting grading state, and its concurrency behavior has been measured rather than assumed. Concretely fixed during this project:

- The `thread_work()` / `pthread_create` signature mismatch that prevented the project from compiling on a modern `gcc` at all.
- An unterminated-buffer write when receiving a student's file (an out-of-bounds read driven entirely by attacker-controlled input), fixed by writing exactly the byte count `recv` returned instead of assuming a `%s`-terminated buffer.
- A latent deadlock in output capture: the original code waited for a child to exit before reading its output pipe, which hangs forever the moment a program prints more than one pipe buffer's worth of output. `capture_with_timeout()` now drains the pipe and waits for the child concurrently.
- No resource limits, no execution timeout, and no upload cap on untrusted code — closed by the sandboxing layer described in Section , verified against five hostile submissions.

Known gaps that remain open in the codebase:

- **Memory leak per test case** — `run()` mallocs a result buffer per test case (`server.c:410`); the pointers stored in `client_results[]` inside `thread_work` are never freed, so the leak grows with every graded submission.
- **Temp files not cleaned up on the `write()` failure path** — if writing the result back to the socket fails, `thread_work` exits early without removing the temporary `.c/.o` files it created.
- **Compile failures are reported as `all-tests-failed`**, not distinctly — a student submission that fails to compile currently just scores 0% with no differentiated feedback, even though `compile()`'s return value already distinguishes the two cases internally.
- **The default thread pool size is still 1000 in `runserver.sh`**, despite Experiment A showing this sits well into the degrading tail of the throughput curve for an 8-core machine; this is a one-line fix that has not yet been applied.

Future work

- **Right-size the default thread pool**, either to a fixed value in the 8–16 range or, better, derived from `nproc` at startup, per the Experiment A recommendation above.
- **Admission control / bounded queue**, so that offered load past the server's saturation point (Experiment B) produces a fast "server busy" response instead of unbounded latency growth.
- **Namespace- or seccomp-based isolation** for filesystem and network confinement of student binaries — the resource-limit layer in Section bounds CPU, memory, process count, and file size, but a compiled student binary can still read/write anywhere the server process can and open arbitrary sockets. This was deliberately scoped out of the current sandboxing pass because it requires root and changes how the server is deployed, which is a different kind of change from adding `setrlimit` calls.
- **Re-run the Experiment A thread-count sweep** now that execution timeouts exist, since a hung-thread failure mode that could previously shrink the effective pool size over time under sustained load is now bounded by `RUN_TIMEOUT_MS`.
- **Compile caching or batched execution** for large test suites, motivated directly by Experiment D's compile/execute crossover: once execution dominates (20+ test cases here), the highest-leverage change is reducing the one-`fork+exec`-per-test-case cost, for example by having the student binary handle all inputs in a single run where the assignment contract allows it.